CEAM GoKart — Sheet Map Area

Full Project Documentation & Source Code

CEAM, MAHE Bengaluru · Oct 2026

1. Project Overview

Field	Value
Project	CEAM GoKart — Autonomous Driving on Sheet Track
Rover	rover1 — Raspberry Pi 5, ROS2 Jazzy
Camera	Intel RealSense D435I (USB 3, 640×480 @ 15 fps)
Current IP	10.235.140.80
Foxglove	ws://10.235.140.80:8765
OS	Ubuntu 22.04
Nav Stack	Nav2 — RegulatedPurePursuit controller

The goal is to build a Bird's Eye View (BEV) map of the sheet track and navigate the rover autonomously via HTTP goal commands from a laptop — no SSH required during demos.

The track is a flat printed plastic sheet with road markings: white lane boundaries, yellow dashed centre lines, white crosswalk stripes, red STOP signs, white direction arrows, and a roundabout. White board walls form the arena boundary.

2. System Architecture

- Pi 5 rover running ROS2 Jazzy
- RealSense D435I mounted facing downward at ~62.6° tilt
- Nav2 (RegulatedPurePursuit) for autonomous navigation
- Foxglove Studio on laptop for visualization via WebSocket
- Flask goal server on rover port 5000 — HTTP commands from laptop
- static_transform_publisher for map→odom (no AMCL / no LiDAR)

3. Session History — What Was Done

3a. Sep 2026 — Nav2 Foundation (AB5 Area)

- Nav2 fully working on Pi 5 rover (Sep 11): map_server + AMCL + RegulatedPurePursuit
- SegFormer-b0 running at ~0.1Hz (seg_node.py), seg_costmap_node.py publishing /seg/costmap + /seg/obstacles
- Obstacle IDs: {0,1,9,14,20} = wall, building, fence, pole, traffic sign

- nav2_custom.launch.py created to avoid bringup_launch.py hardcoding issues
- Static transform approach discovered — bypasses AMCL entirely, rover moves straight

3b. Oct 4–6 — Nav2 Debugging

- All Nav2 nodes active for first time Oct 4; nav goal accepted; rover moved forward
- AMCL consistently failing to converge — covariance stuck at ~0.25
- Oct 6: Static transform (map→odom) confirmed working — rover moves straight, no spinning
- seg_layer detected bottle obstacle briefly
- collision_monitor crashing with invalid type error on FootprintApproach.points — fixed by removing points:1000, keeping only min_points:6

3c. Oct 8 — Track Area Move

- Moved to new sheet map arena (plastic printed track, white board walls)
- slam_toolbox installed; SLAM attempted but LiDAR map too noisy on flat plastic surface
- Decision: abandon LiDAR SLAM for this track, pivot to camera BEV

3d. Oct 13–14 — BEV Development (Depth-Based, USB 3 Fix)

- Lead confirmed: face camera down, use Lift Splat Shoot BEV
- Camera intrinsics confirmed: fx=607.57, fy=607.52, cx=323.53, cy=249.92 @ 640x480
- USB 2 → USB 3 cable swap: RealSense now at 5000M on Bus 003
- RGB stable at 14.75 FPS, aligned depth at 15.0 FPS after killing seg_node
- Camera calibrated via depth plane fit: height=0.153m, tilt=40.1°
- numpy downgraded to 1.26.4 in seg_venv (cv_bridge conflict with numpy 2.x)
- Depth decoded via np.frombuffer to bypass cv_bridge numpy conflict
- bev_node.py built: manual timestamp sync, BEST_EFFORT QoS — publishing at ~1-5 FPS
- BEV image visible in Foxglove but mostly black — depth too sparse near ground on USB 2.1

3e. Oct 17–19 — Homography BEV + Goal Server (Current Working State)

- SWITCHED to RGB homography BEV (bev_map_node.py) — depth approach abandoned for USB 2.1
- cv2.getPerspectiveTransform with 4 manually calibrated ground corners → 400×400 top-down canvas
- Key fix: publisher QoS MUST be BEST_EFFORT — Foxglove image panels reject RELIABLE topics
- Key fix: Foxglove Color mode MUST be RGB (Gradient renders black)
- BEV image now fully visible in Foxglove showing track surface ✅
- goal_server.py: Flask HTTP server on rover port 5000 — laptop sends curl commands
- Autonomous navigation confirmed working via curl goal commands
- Known issue: rover spins excessively — max_angular_vel needs reducing to 0.5

4. BEV Implementation

4a. Why NOT Depth-Based BEV

The D435I on USB 2.1 produces mostly invalid depth for near-ground pixels. The depth-based node (bev_node.py) gave a mostly black image — valid depth pixels were too sparse on the ground plane. bev_node.py is kept for reference only.

4b. Homography BEV — bev_map_node.py (ACTIVE)

RGB-only approach using cv2.getPerspectiveTransform. Four ground-plane corners in the 640×480 camera image are warped to a 400×400 top-down canvas. No depth data needed.

Corner	Camera pixel (src)	BEV pixel (dst)
Bottom-left	(160, 480)	(50, 380)
Bottom-right	(480, 480)	(350, 380)
Top-right	(540, 280)	(350, 50)
Top-left	(100, 280)	(50, 50)

Camera: RealSense D435I, 640×480, tilt ~62.6°, height ~0.147m above track. Output canvas: 400×400 px, rgb8 encoding.

QoS: BEST_EFFORT on both subscriber and publisher. Foxglove panel: Color mode → RGB.

4c. Camera Calibration Parameters

Parameter	Value
fx	607.5739
fy	607.5168
cx	323.5252
cy	249.9227
Resolution	640×480
Tilt angle	~62.6° downward from horizontal
Height above track	~0.147m
Distortion model	plumb_bob (d = [0,0,0,0,0])

5. Goal Server — Remote Navigation

Flask HTTP server on rover port 5000. Laptop sends curl commands — no SSH needed during demos.

Method	Endpoint	Body	Action

GET	/status	—	Health check
POST	/goal	{"x":1.0,"y":0.5,"yaw":0.0}	Send single Nav2 goal
POST	/waypoints	{"points":[{"x":1,"y":0},{"x":2,"y":1}]}	Sequential waypoints (3s apart)
POST	/stop	—	Send goal (0,0,0) to stop

Example commands from laptop:

```
curl http://10.235.140.80:5000/status
curl -X POST http://10.235.140.80:5000/goal \
  -H "Content-Type: application/json" \
  -d '{"x":1.0,"y":0.5,"yaw":0.0}'
curl -X POST http://10.235.140.80:5000/stop
```

If port 5000 is busy: fuser -k 5000/tcp

6. Nav2 Setup

Component	**Value**
Global planner	NavFn (NavfnPlanner)
Local controller	RegulatedPurePursuit
Config file	~/nav2_params.yaml
Map → odom	Static transform (no AMCL, no LiDAR)
Lifecycle manager	autostart:true in nav2_custom.launch.py

Static transform command:

```
ros2 run tf2_ros static_transform_publisher \
  --frame-id map --child-frame-id odom \
  --x 0 --y 0 --z 0 --yaw 0 --pitch 0 --roll 0
```

7. Startup Sequence (Each Session)

SSH into rover first for every terminal:

```
ssh rover1@10.235.140.80
```

Terminal	**Command**	**Notes**
T1	ros2 launch rover_description bringup.launch.py	Base robot description
T2	pkill -f ydlidar	Kill LiDAR — not needed
T3	ros2 launch realsense2_camera rs_launch.py rgb_camera.color_profile:=640x480x15 depth_module.depth_profile:=640	USB 3 cable required

	x480x15 pointcloud.enable:=false align_depth.enable:=true	
T4	ros2 launch ~/depthscan.launch.py	"No camera info" initially normal
T5	pkill -f ekf_filter_node && sleep 2 && ros2 launch rover_description ekf_odom.launch.py	Kill duplicates first
T6a	ros2 run tf2_ros static_transform_publisher --frame-id map --child-frame-id odom --x 0 --y 0 --z 0 --yaw 0 --pitch 0 --roll 0	Replaces AMCL
T6b	ros2 launch ~/nav2_custom.launch.py	Nav2 stack
T7	ros2 launch foxglove_bridge foxglove_bridge_launch.xml	Connect at ws://10.235.140.80:8765
T8	source ~/seg_venv/bin/activate && python3 ~/bev_map_node.py	BEV node — DO NOT run seg_node.py
T9	source ~/seg_venv/bin/activate && python3 ~/goal_server.py	Flask goal server

Foxglove: Add Image panel → topic /bev/image → Color mode: RGB

8. Known Issues & Fixes

Issue	Cause	Fix
Foxglove "Waiting for image messages"	QoS mismatch — publisher was RELIABLE	Use BEST_EFFORT QoS in publisher
BEV all black	D435I USB 2.1 near-field depth invalid	Switch to RGB homography (bev_map_node.py)
Foxglove color wrong (dark)	Color mode set to Gradient	Change to RGB in panel settings
Port 5000 busy	Old flask server still running	fuser -k 5000/tcp
Rover spinning excessively	max_angular_vel too high	Reduce to 0.5 in nav2_params.yaml under FollowPath
CPU overload / camera drops	seg_node.py pegs CPU at 100%	Do NOT run seg_node.py or seg_costmap_node.py with BEV
RealSense 7-8 FPS	USB 2.1 bandwidth limitation	Use USB 3 Type-A-to-Type-C cable

9. Pending Work

#	Task	Status	Notes

1	Fix rover spinning	In progress	Reduce max_angular_vel to 0.5 in nav2_params.yaml
2	Whiteboard boundary detection	Not started	HSV white mask: H:0-180, S:0-50, V:200-255 → publish as Nav2 obstacles
3	Global track map	Not started	Options: hand-draw .pgm, SLAM, or stitch BEV frames
4	BEV → Nav2 costmap layer	Not started	Feed /bev/image into Nav2 local costmap as obstacle layer
5	Lift Splat Shoot BEV	Future	Lead recommendation — needs proper USB 3 depth pipeline
6	flask install check	Pending	pip install flask --break-system-packages on rover if not installed

10. File Reference

File	Status	Purpose
bev_map_node.py	ACTIVE	RGB homography BEV — publishes /bev/image
goal_server.py	ACTIVE	Flask HTTP server for remote Nav2 goal delivery
bev_node.py	Reference only	Depth-based BEV — not working on USB 2.1
nav2_custom.launch.py	ACTIVE	Custom Nav2 launch file
nav2_params.yaml	ACTIVE	Nav2 parameters (RegulatedPurePursuit)
depthscan.launch.py	ACTIVE	Depth image to laser scan
seg_node.py	Inactive	SegFormer-b0 semantic segmentation (kills CPU on Pi5)
seg_costmap_node.py	Inactive	Segmentation costmap publisher

11. Source Code

11a. bev_map_node.py — RGB Homography BEV (ACTIVE)

```
#!/usr/bin/env python3
"""
BEV Map Node — CEAM GoKart Project, MAHE Bengaluru
RGB homography BEV — no depth needed.
Camera: RealSense D435I, 640x480, tilt ~62.6 deg
Publishes: /bev/image (rgb8, 400x400)
Run: source ~/seg_venv/bin/activate && python3 ~/bev_map_node.py
"""
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
from sensor_msgs.msg import Image
import numpy as np
import cv2

class BEVMapNode(Node):
    def __init__(self):
        super().__init__('bev_map_node')
        # BEST_EFFORT required — Foxglove rejects RELIABLE image topics
        qos = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1)
        self.sub = self.create_subscription(
            Image, '/camera/camera/color/image_raw', self.cb, qos)
        self.pub = self.create_publisher(Image, '/bev/image', qos)
        # src: 4 ground corners in 640x480 image — tune to your mount
        src = np.float32([
            [160, 480],  # bottom-left
            [480, 480],  # bottom-right
            [540, 280],  # top-right
            [100, 280],  # top-left
        ])
        self.OUT_W, self.OUT_H = 400, 400
        dst = np.float32([
            [50,  380], [350, 380], [350, 50], [50, 50],
        ])
        self.H_mat = cv2.getPerspectiveTransform(src, dst)
        self.get_logger().info('BEV map node ready — publishing /bev/image')

    def cb(self, msg):
        img = np.frombuffer(msg.data, dtype=np.uint8).reshape(
            msg.height, msg.width, 3)
        bev = cv2.warpPerspective(img, self.H_mat, (self.OUT_W, self.OUT_H))
        cv2.circle(bev, (self.OUT_W//2, self.OUT_H-20), 8, (0,255,0), -1)
        out = Image()
        out.header   = msg.header
        out.height   = self.OUT_H
```

```
        out.width    = self.OUT_W
        out.encoding = 'rgb8'
        out.step     = self.OUT_W * 3
        out.data     = bev.tobytes()
        self.pub.publish(out)

def main():
    rclpy.init()
    node = BEVMapNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

11b. goal_server.py — Flask Goal Server (ACTIVE)

```
#!/usr/bin/env python3
"""
Goal Server — CEAM GoKart, MAHE Bengaluru
Flask HTTP server on rover port 5000.
Run: source ~/seg_venv/bin/activate && python3 ~/goal_server.py
If port busy: fuser -k 5000/tcp
"""
import threading, math
from flask import Flask, request, jsonify
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
from geometry_msgs.msg import PoseStamped

app = Flask(__name__)
ros_node = None

class GoalSender(Node):
    def __init__(self):
        super().__init__('goal_server')
        self._client = ActionClient(self, NavigateToPose,
'/navigate_to_pose')

    def send_goal(self, x, y, yaw=0.0):
        self._client.wait_for_server()
        goal = NavigateToPose.Goal()
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.header.stamp = self.get_clock().now().to_msg()
        pose.pose.position.x = float(x)
        pose.pose.position.y = float(y)
        pose.pose.orientation.z = math.sin(yaw / 2)
        pose.pose.orientation.w = math.cos(yaw / 2)
        goal.pose = pose
        self._client.send_goal_async(goal)

@app.route('/status', methods=['GET'])
def status():
    return jsonify({'status': 'online', 'rover': 'rover1'})

@app.route('/goal', methods=['POST'])
def receive_goal():
    data = request.json
    ros_node.send_goal(data.get('x',0.0), data.get('y',0.0),
data.get('yaw',0.0))
    return jsonify({'status': 'goal sent'})

@app.route('/waypoints', methods=['POST'])
def waypoints():
    points = request.json.get('points', [])
```

```
    def run():
        import time
        for pt in points:
            ros_node.send_goal(pt['x'], pt['y'], pt.get('yaw', 0.0))
            time.sleep(3)
    threading.Thread(target=run, daemon=True).start()
    return jsonify({'status': 'waypoints sent', 'count': len(points)})

@app.route('/stop', methods=['POST'])
def stop():
    ros_node.send_goal(0.0, 0.0, 0.0)
    return jsonify({'status': 'stop sent'})

def spin_ros():
    rclpy.spin(ros_node)

if __name__ == '__main__':
    rclpy.init()
    ros_node = GoalSender()
    threading.Thread(target=spin_ros, daemon=True).start()
    app.run(host='0.0.0.0', port=5000)
```

11c. nav2_custom.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='nav2_controller', executable='controller_server',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_planner', executable='planner_server',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_behaviors', executable='behavior_server',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_bt_navigator', executable='bt_navigator',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_velocity_smoother',
executable='velocity_smoother',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_collision_monitor',
executable='collision_monitor',
             output='screen', parameters=['/home/rover1/nav2_params.yaml']),
        Node(package='nav2_lifecycle_manager',
executable='lifecycle_manager',
             output='screen', parameters=[{
                 'use_sim_time': False,
                 'autostart': True,
                 'node_names': [
                     'controller_server', 'planner_server',
                     'behavior_server', 'bt_navigator',
                     'velocity_smoother', 'collision_monitor',
                 ]
             }]),
    ])
```

11d. depthscan.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='depthimage_to_laserscan',
            executable='depthimage_to_laserscan_node',
            name='depthimage_to_laserscan',
            remappings=[
                ('depth', '/camera/camera/depth/image_rect_raw'),
                ('depth_camera_info', '/camera/camera/depth/camera_info'),
            ],
            parameters=[{
                'output_frame': 'base_link',
                'range_min': 0.3,
                'range_max': 4.0,
                'scan_height': 10,
            }],
            ros_arguments=[
                '--ros-args',
                '--param',
'qos_overrides./camera/camera/depth/camera_info.subscription.reliability:=be
st_effort',
                '--param',
'qos_overrides./camera/camera/depth/image_rect_raw.subscription.reliability:
=best_effort',
            ]
        )
    ])
```

11e. bev_node.py — Depth-Based BEV (Reference Only — Not Working on USB 2.1)

This was the original depth-based LSS-style BEV node. It is kept for reference. The depth approach was abandoned because the D435I on USB 2.1 produces too few valid ground-plane depth pixels, resulting in a mostly black BEV image. Use bev_map_node.py instead.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
from sensor_msgs.msg import Image
from nav_msgs.msg import OccupancyGrid
from cv_bridge import CvBridge
import numpy as np, cv2

class BEVNode(Node):
    def __init__(self):
        super().__init__('bev_node')
        self.bridge = CvBridge()
        self.latest_rgb = self.latest_depth = None
        self.latest_rgb_stamp = self.latest_depth_stamp = None
        self.last_proc_time = 0.0
        self.frame_count = 0
        # Calibrated extrinsics (Oct 14 2026)
        self.cam_height   = 0.147   # metres
        self.cam_tilt_deg = 62.6    # degrees downward
        # Half-res intrinsics (320x240 after resize)
        self.fx, self.fy  = 303.787, 303.759
        self.cx, self.cy  = 161.763, 124.962
        self.bev_res = 0.005; self.bev_width = 300; self.bev_height = 200
        qos = QoSProfile(reliability=QoSReliabilityPolicy.BEST_EFFORT,
                         history=QoSHistoryPolicy.KEEP_LAST, depth=1)
        pub_qos = QoSProfile(reliability=QoSReliabilityPolicy.RELIABLE,
                             history=QoSHistoryPolicy.KEEP_LAST, depth=1)
        self.create_subscription(Image,
            '/camera/camera/color/image_raw', self.rgb_cb, qos)
        self.create_subscription(Image,
            '/camera/camera/aligned_depth_to_color/image_raw',
self.depth_cb, qos)
        self.pub_img = self.create_publisher(Image, '/bev/image', pub_qos)
        self.pub_bev = self.create_publisher(OccupancyGrid, '/bev/costmap',
pub_qos)

    def rgb_cb(self, msg):
        self.latest_rgb = msg
        self.latest_rgb_stamp = msg.header.stamp.sec +
msg.header.stamp.nanosec*1e-9
        self.try_process()

    def depth_cb(self, msg):
        self.latest_depth = msg
        self.latest_depth_stamp = msg.header.stamp.sec +
msg.header.stamp.nanosec*1e-9
```

```
        self.try_process()

    def try_process(self):
        if self.latest_rgb is None or self.latest_depth is None: return
        if abs(self.latest_rgb_stamp - self.latest_depth_stamp) > 0.5:
return
        now = self.latest_rgb_stamp
        if now - self.last_proc_time < 0.2: return
        self.last_proc_time = now
        self.process(self.latest_rgb, self.latest_depth)

    def process(self, rgb_msg, depth_msg):
        rgb = self.bridge.imgmsg_to_cv2(rgb_msg, 'rgb8')
        rgb = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
        rgb = cv2.resize(rgb, (320, 240))
        depth = np.frombuffer(depth_msg.data, dtype=np.uint16).reshape(
                    depth_msg.height,
depth_msg.width).astype(np.float32)/1000.0
        depth = cv2.resize(depth, (320, 240))
        h, w = depth.shape
        uu, vv = np.meshgrid(np.arange(w), np.arange(h))
        Z = depth; valid = (Z > 0.05) & (Z < 4.0)
        X_c = (uu - self.cx) * Z / self.fx
        Y_c = (vv - self.cy) * Z / self.fy
        tilt = np.radians(self.cam_tilt_deg)
        ct, st = np.cos(tilt), np.sin(tilt)
        Y_w = Y_c*ct - Z*st + self.cam_height
        X_w = X_c; Z_w = Y_c*st + Z*ct
        gnd = valid & (np.abs(Y_w)<0.30) & (Z_w>0.01) &
(Z_w<self.bev_height*self.bev_res)
        bev = np.full((self.bev_height, self.bev_width, 3), 30,
dtype=np.uint8)
        bx = ((X_w+self.bev_width*self.bev_res/2)/self.bev_res).astype(int)
        by = (self.bev_height-1-(Z_w/self.bev_res)).astype(int)
        ok = gnd&(bx>=0)&(bx<self.bev_width)&(by>=0)&(by<self.bev_height)
        bev[by[ok], bx[ok]] = rgb[vv[ok], uu[ok]]
        # ... (HSV segmentation and OccupancyGrid publishing omitted for
brevity)
        self.frame_count += 1

def main():
    rclpy.init(); rclpy.spin(BEVNode()); rclpy.shutdown()
if __name__ == '__main__': main()
```